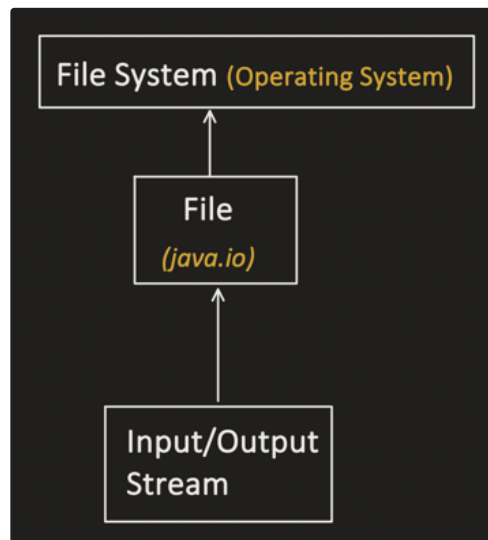


File Handling (Traditional)



In File:-

File file = new File("Sample.txt");

This does not:

- *Open file*
- *Read file*
- *Write file*

It just represent filesystem path and provides helper methods to interact with that path via the OS:

- *Path helper method:*
 - *file.getName();*
 - *file.getAbsolutePath();*
- *Metadata helper methods:*
 - *file.exists();*
 - *file.isDirectory();*

- `file.lastModified();`
- File Operation helper methods:
 - `file.createNewFile();`
 - `file.delete();`
 - `file.mkdir();`

In Input/Output stream:-

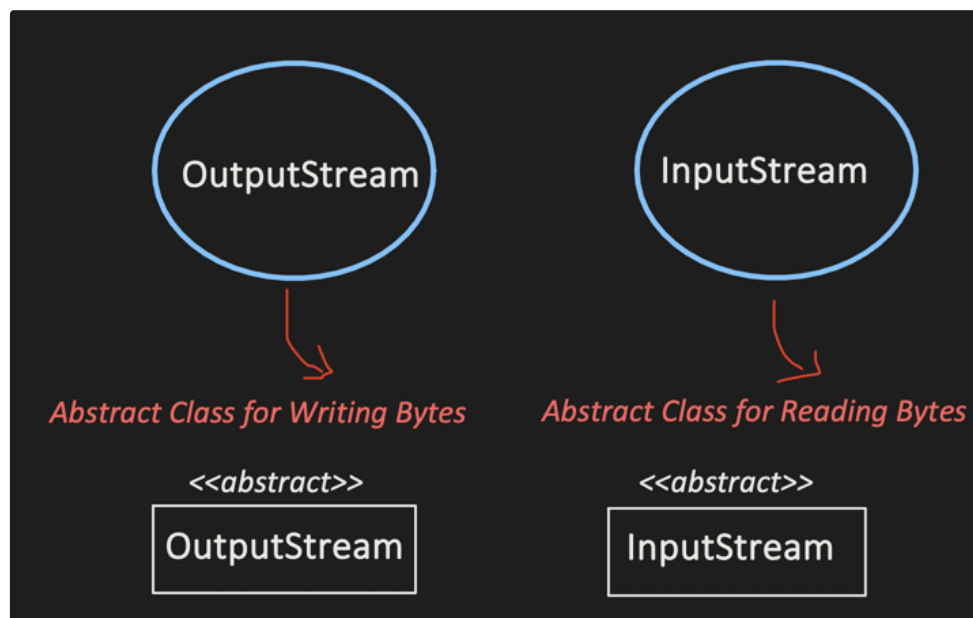
It is the one which actually, makes OS system calls to:

- Open
- Read and
- Write files

Considered Stream as bridge to OS to manage file.

Input/Output Stream internally make use of Decorator Design Pattern.

1st Layer:



At the lowest level, no matter we are reading or writing Characters, Integers, Image, Object etc.

Everything happens in bytes only.

- Character → encoded → bytes
- Integer → binary → bytes
- Object → serialized → bytes

etc.

Implementation of these Abstract classes:



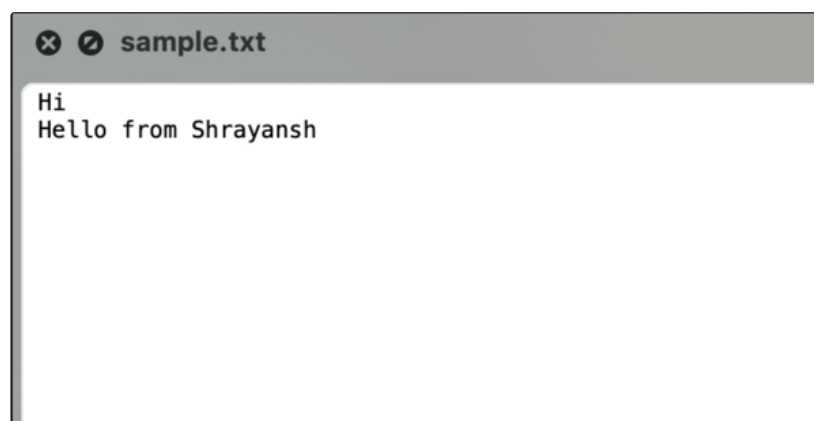
```
1 public class ByteStreams {
2
3
4     public static void main(String[] args) throws IOException {
5         /*
6          Internally makes OS sys call : Open(path, O_TRUNC)
7
8          If file exists -> Content is deleted
9          If not -> new file is created
10
11         If we want append behavior:
12
13         new FileOutputStream("sample.txt", true);
14
15         This will append to existing file and internally makes OS sys
16         call :
17         Open(path, O_APPEND)
18         */
19         FileOutputStream fos = new FileOutputStream("sample.txt");
20
21         //Writing byte by byte to a file
22         fos.write(72); // 'H' in ASCII
23         fos.write(105); // 'i' in ASCII
24         fos.write('\n'); // newline
25
26         // writes an entire byte array at once
27         byte[] message = "Hello from Shrayansh".getBytes();
28         fos.write(message);
29         /*
30         Always make sure that stream is closed once work is done bcoz:
31
32         - Internally stream works with OS and OS don't directly touch
33         disk,
34         it also maintains buffer (kernel buffer), so that need to be
35         flushed to Disk.
```

```

35     - Make stream unusable, means once closed same stream can not be
      used
36     for write or read operation again.
37     etc.
38
39     */
40
41     fos.close();
42
43     System.out.println("File written successfully.");
44
45
46
47
48
49     // Reading byte from a file
50     File file = new File("sample.txt");
51     FileInputStream fis = new FileInputStream(file);
52
53     if(!file.exists()) {
54         // file do not exists
55         return;
56     }
57
58     // Reading byte by byte
59     int byteValue;
60     /*
61     Fis read() :
62     returns byte value -> 0-255
63     Or
64     -1 if there is no byte i.e (EOF)
65     */
66     while ((byteValue = fis.read()) != -1) {
67         System.out.print((char) byteValue);
68     }
69
70     fis.close();
71
72 }
73 }
74

```

Output: File Write



Output: File Read

```
--- Reading byte by byte ---  
Hi  
Hello from Shrayansh
```

2nd Layer: Decorator Pattern

- While using FileOutputStream each "**write()**" is a system call and call OS.
- Similarly, while using FileInputStream each "**read()**" is a system call and call OS.

And during each **SYSTEM call**, our Java program **STOPS** and switches to OS kernel mode.

OS read/write from disk, switches back to our code.

This context switch takes time.

Just assume that file with 10Lakh bytes and we are reading byte by byte.

So 10Lakh SYSTEM Call → could lead to several seconds of overhead.

SOLUTION: BufferedInputStream

Without buffering:

read() → [syscall] → OS → disk → 1byte

(for file with 10Lakh bytes : ×1,000,000 times syscall)

With buffering:

read() → buffer empty → [syscall] → OS → disk → 8192bytes (default)

read() → buffer hit → 1byte(no syscall, just memory)

read() → buffer hit → 1byte

...

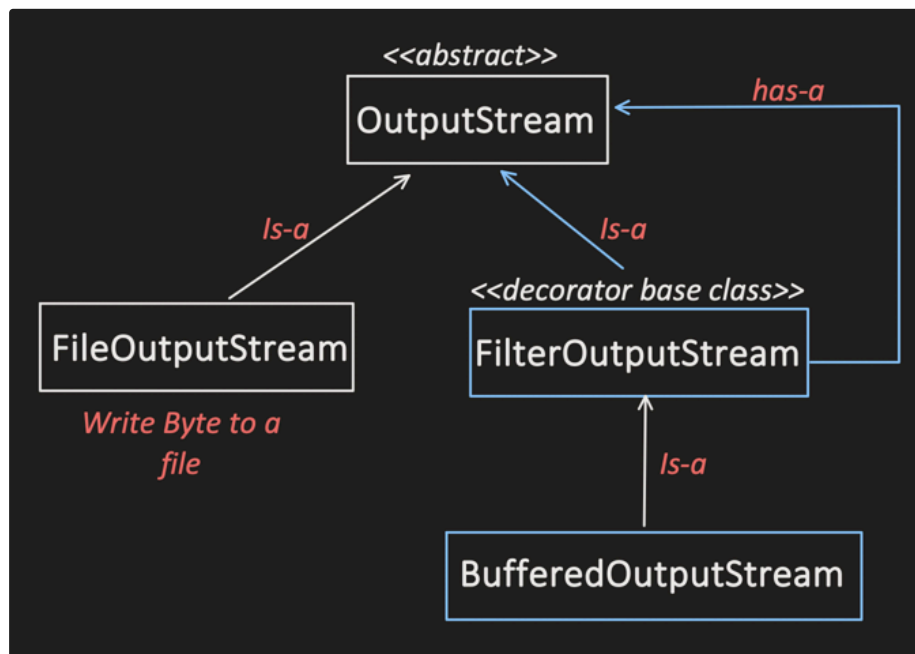
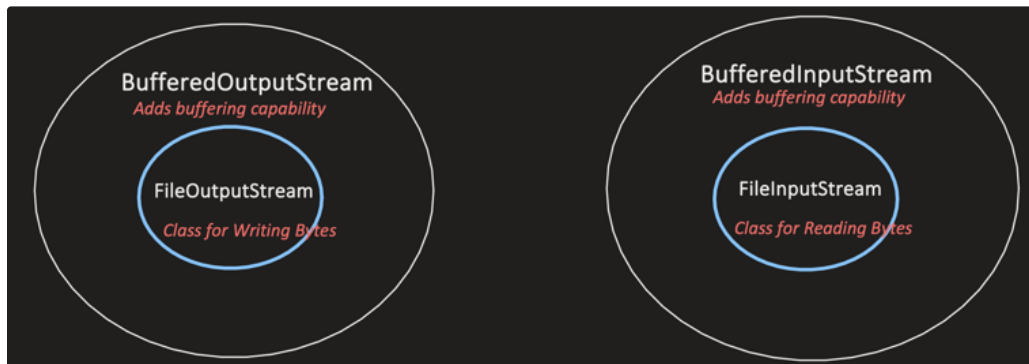
... 8192 bytes reads from buffer...

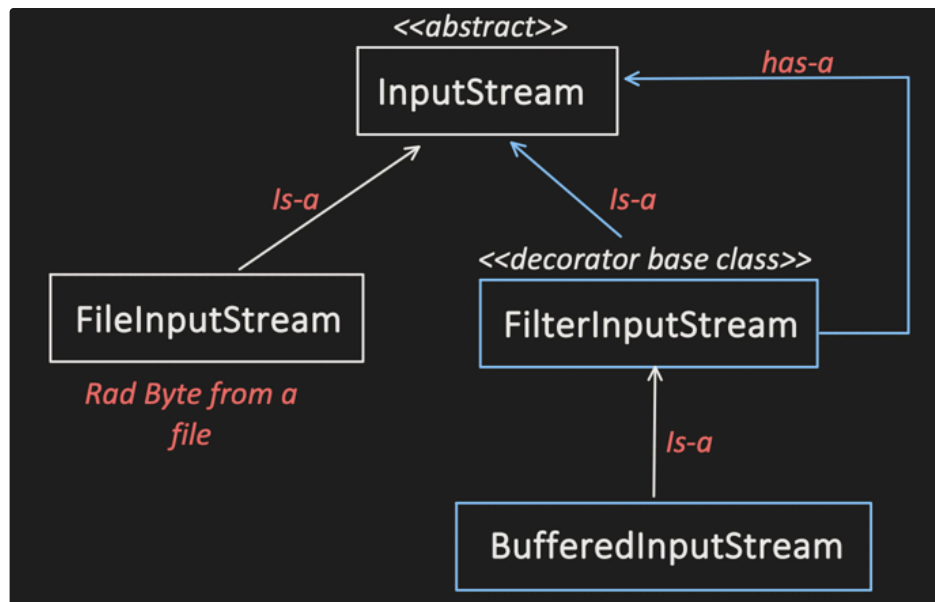
`read()` → buffer empty → [syscall] → OS → disk → another 8192bytes

`read()` → buffer hit → 1byte

... (~122 syscalls total)

And `BufferedInputStream` / `BufferedOutputStream` classes are good example of Decorator design pattern.





```

1 public class Buffering{
2
3     public static void main(String[] args) throws IOException {
4
5         //Without buffering, every write() goes to OS immediately
6         //with buffering: writes accumulated in memory and flushed in
7         big chunks.
8         BufferedOutputStream bos = new BufferedOutputStream(new
9         FileOutputStream("sample.txt"));
10
11         for(int i=0 ; i<1000; i++) {
12             /*
13              Internally it stored as bytes only.
14
15              'L' → 76
16              'i' → 105
17              'n' → 110
18              'e' → 101
19              ' ' → 32
20              '1' → 49
21              '2' → 50
22              '\n' → 10
23              All are single byte values.
24              */
25             bos.write(("Line " + i + "\n").getBytes());
26             //These go to the internal buffer, NOT to disk yet.
27         }
28
29         // this close, also flushes the write from buffer to disk too.
30         bos.close();
31
32         System.out.println("File written successfully.");
33     }
  
```

```

1 //we WRAP FileInputStream inside BufferedInputStream.
2 //BufferedInputStream does NOT replace FileInputStream.
3 //It ADDS buffering ON TOP of it. Same API -> just faster.
  
```

```

4      /*
5         Default is 8192 bytes.
6         But if we want custom, we can
7         also change it:
8
9         new BufferedInputStream (
10            new FileInputStream("sample.txt"), 16384);
11      */
12
13      BufferedInputStream bis= new BufferedInputStream (new
14      FileInputStream("sample.txt"));
15
16      // Reading byte by byte but from buffer first and when buffer is empty
17      // then only call goes to OS
18      int byteValue;
19      while ((byteValue = bis.read()) != -1) {
20          System.out.print((char) byteValue);
21      }
22
23      bis.close();
24
25  }
26

```

SPEED COMPARISION:

```

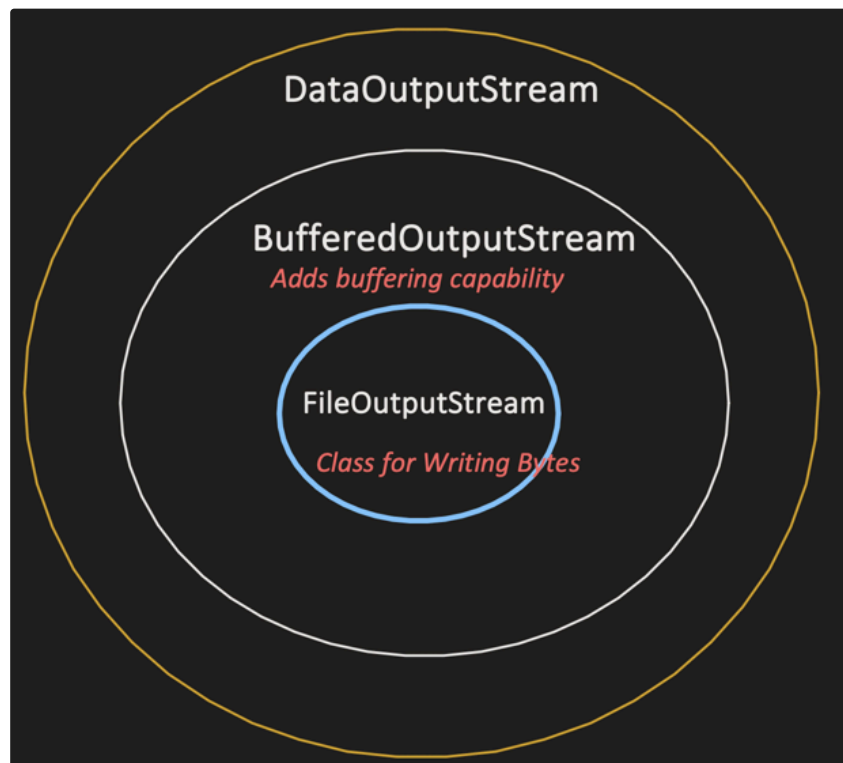
1  public static void main(String[] args) throws IOException {
2
3      // creates testfile.txt with 50,000 bytes
4      FileOutputStream fos = new FileOutputStream("testfile.txt");
5      for (int i = 0; i < 50000; i++) {
6          fos.write('A' + (i % 26));
7      }
8      fos.close();
9
10     //speed without buffering
11     long start1 = System.nanoTime();
12     FileInputStream fis = new FileInputStream("testfile.txt");
13     while (fis.read() != -1) {
14         /* just consume */
15     }
16     fis.close();
17     long end1 = System.nanoTime();
18     System.out.println("Without buffering: " + (end1 - start1) /
19     1_000_000 + " ms");
20
21     long start2 = System.nanoTime();
22     BufferedInputStream bis = new BufferedInputStream(new
23     FileInputStream("testfile.txt"));
24     while (bis.read() != -1) {
25         /* just consume */
26     }
27     bis.close();
28     long end2 = System.nanoTime();
29     System.out.println("With buffering: " + (end2 - start2) /
30     1_000_000 + " ms");
31 }

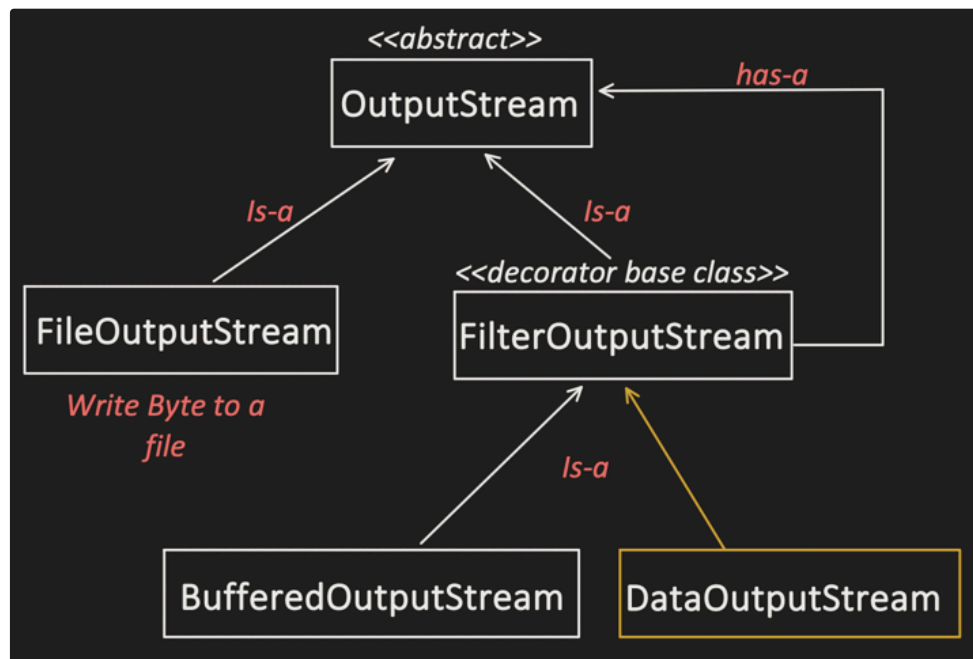
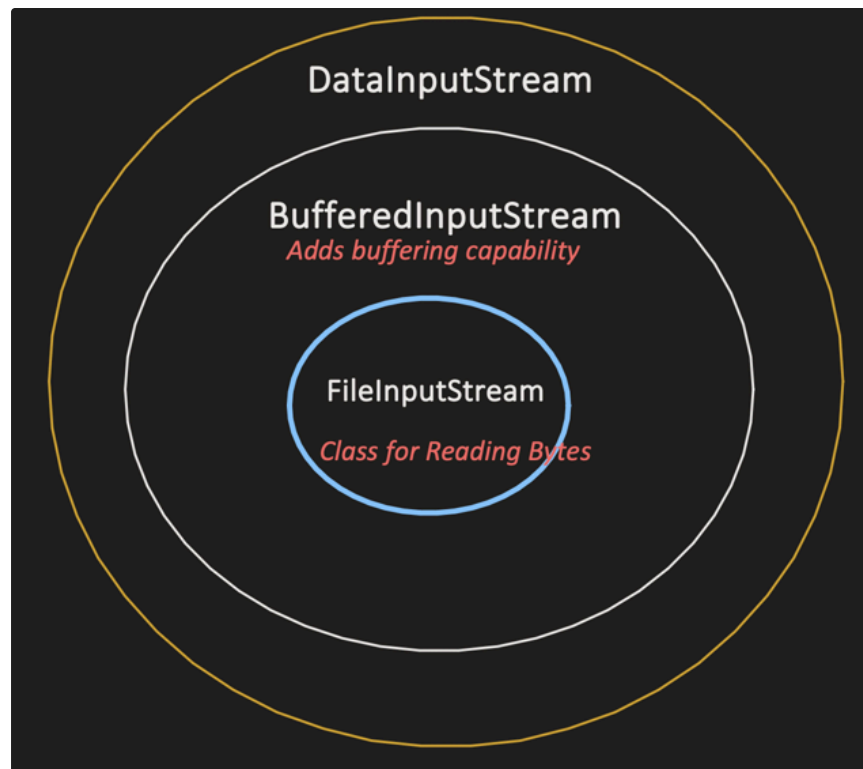
```

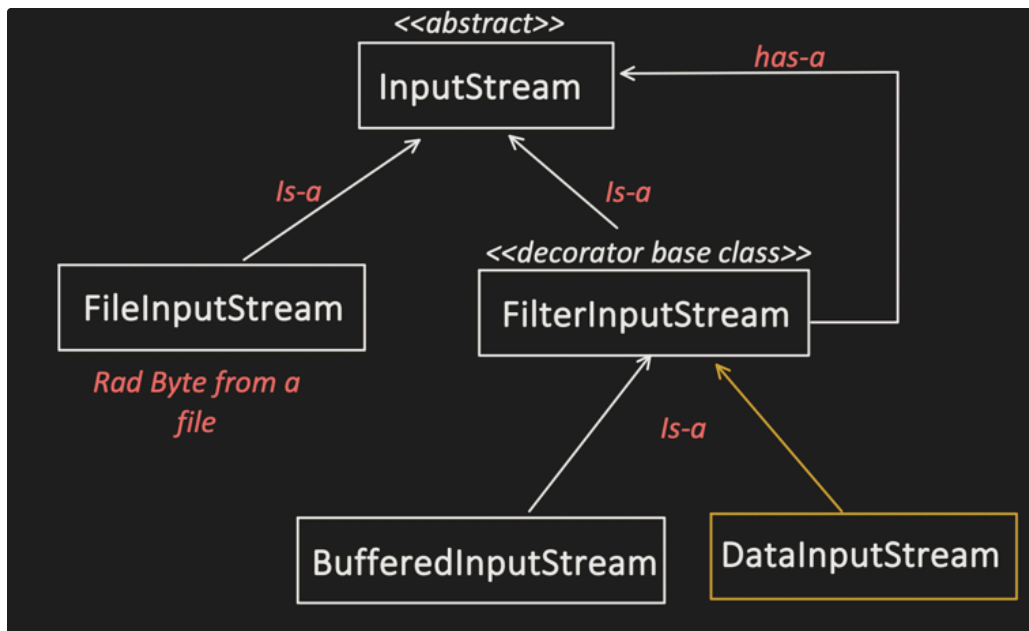

Output:

```
Without buffering: 20 ms  
With buffering:    3 ms  
  
Process finished with exit code 0
```

We can add many layers now:







FileOutputStream/FileInputStream → Deals with Bytes

Till then we are writing and reading from file and do not see issues because every character we used can be represented using 1 byte.

Problem: what if we want to store "int var = 300"

Now int, represented by 4 bytes

300 = 00000000 00000000 00000001 00101100

So how to write:

```
fos.write(0);//byte1
```

```
fos.write(0);//byte2
```

```
fos.write(1);//byte3
```

```
fos.write(44);//byte4
```

And how to read:

We have to read 4 bytes and then reassemble (which is very tedious task).

Similarly, what if we want to store: long, double, boolean etc.

Solution: DataOutputStream / DataInputStream

For primitive data types, it handles write and read for us.

DataOutputStream / DataInputStream, just tell how to break primitive data type into bytes and rebuild them back.

Ex:

Break int 300 into 4 bytes in a fixed order

[00][00][01][2C] → write to file

Read 4 bytes → combine → 300

```
1 public class DataStreams {
2
3     public static void main(String[] args) throws IOException {
4
5         // Imagine saving a "Player" record: name, score, health,
6         // isAlive
7         /*
8          * This is optional, you can decorate Output stream with alone
9          * Data Stream or with both Data Stream and Buffered Stream.
10         */
11         /*
12          *
13          * DataOutputStream dos = new DataOutputStream(new
14          * BufferedOutputStream(new FileOutputStream("player.txt")));
15
16          * dos.writeInt(9500); // player id, exactly 4 bytes
17          * dos.writeDouble(87.5); // health, exactly 8 bytes
18          * dos.writeBoolean(true); // isAlive, exactly 1 byte
19          * dos.writeLong(1711234567890L); // timestamp, exactly 8
20          * bytes
21          * dos.writeChar('A'); // rank, exactly 2 bytes
22          * (UTF-16)
23          * dos.close();
24          * System.out.println("Player data written.");
25
26          * // MUST read in the EXACT SAME ORDER as written.
27          * // If we read an int where a double was written, we get
28          * garbage value.
29
30          * DataInputStream dis = new DataInputStream(new
31          * BufferedInputStream(new FileInputStream("player.txt")));
32          * int id = dis.readInt(); // reads 4 bytes -> int
33          * double health = dis.readDouble(); // reads 8 bytes ->
34          * double
35          * boolean isAlive = dis.readBoolean(); // reads 1 byte ->
36          * boolean
37         */
38     }
39 }
```

```

30         long timestamp = dis.readLong();           // reads 8 bytes ->
31     long
32         char rank = dis.readChar();                 // reads 2 bytes -> char
33     dis.close();
34     System.out.println(" Player Record ");
35     System.out.println("PlayerId: " + id);
36     System.out.println("Health: " + health);
37     System.out.println("Alive: " + isAlive);
38     System.out.println("Timestamp: " + timestamp);
39     System.out.println("Rank: " + rank);
40 }
41 }
42

```

PROBLEM:

DataOutputStream lets us write primitives field by field. But what if we have a whole object with 10 fields?

we have to write each field manually and read them back in order.

SOLUTION: Serialization along with **ObjectOutputStream/ObjectInputStream**

Java can automatically convert an entire object (and all objects it references) into a stream of bytes, and reconstruct it later.

This is called serialization (object → bytes) and deserialization (bytes → object).

HOW TO MAKE A CLASS SERIALIZABLE: Just implement the Serializable interface.

Object with Serialization

```

1  class Player implements Serializable {
2
3      String name;
4      int score;
5      double health;
6      // transient, will NOT be saved to file
7      transient String sessionToken;
8      // static fields are NOT serialized either (they belong to the
9      // class, not instance)
10     static int totalPlayers = 0;
11
12     Player(String name, int score, double health, String sessionToken)
13     {
14         this.name = name;
15         this.score = score;
16         this.health = health;
17     }
18 }

```

```

15         this.sessionToken = sessionToken;
16         totalPlayers++;
17     }
18 }
19

```

ObjectOutputStream/ObjectInputStream

```

1 public class Serialization {
2
3     public static void main(String[] args) throws IOException,
4     ClassNotFoundException {
5
6         Player player1 = new Player("Shrayansh", 9500, 87.5,
7         "tok_abc123");
8
9         ObjectOutputStream oos = new ObjectOutputStream(new
10        BufferedOutputStream(
11        new FileOutputStream("player.txt")));
12
13        oos.writeObject(player1); // entire object → bytes → file
14        oos.close();
15
16        //Deserialize it back
17        ObjectInputStream ois = new ObjectInputStream(
18        new
19        BufferedInputStream(new FileInputStream("player.txt"))); {
20
21        // readObject() returns Object, must cast to the right type
22        Player loaded = (Player) ois.readObject();
23    }
24 }
25

```

Now last but important:

How to handle characters?

You might say, we already did with **FileOutputStream** and **FileInputStream**

For ex: we did this

```

1 // Reading byte from a file
2 File file = new File("sample.txt");
3 FileInputStream fis = new FileInputStream(file);
4
5 if(!file.exists()) {
6     // file do not exists
7     return;
8 }
9
10 // Reading byte by byte
11 int byteValue;
12 /*

```

```

13      This code is properly showing the file content because:
14
15      - Whatever we wrote in file that's plain English (and each
    character
16      can be represented by 1 byte)
17
18      - And when we read it, we read byte by byte, so each byte properly
19      converted to char.
20      */
21  while ((byteValue = fis.read()) != -1) {
22      System.out.print((char) byteValue);
23  }
24
25  fis.close();
26

```

But problem is: **TEXT has ENCODING**

Example:

\$ sign:

- UTF-8 format: represented in 3 bytes
- UTF-16 format: represented in 2 bytes

etc.

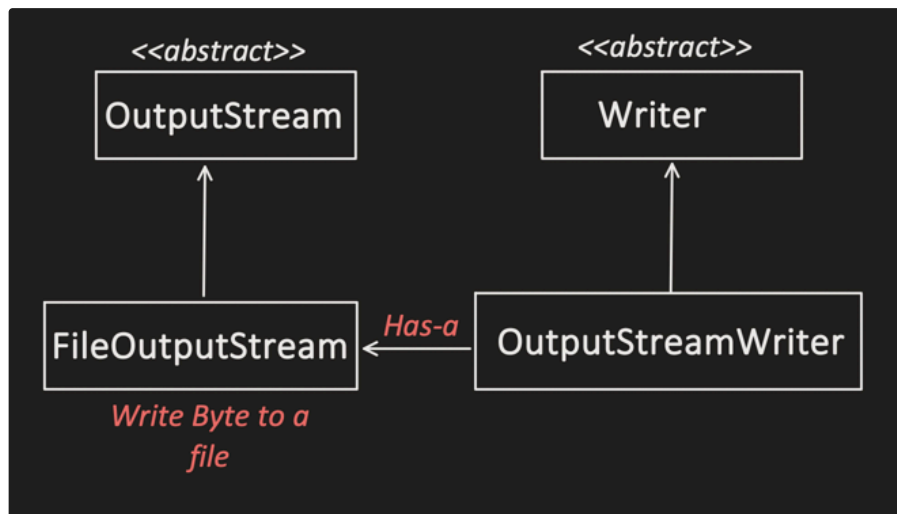
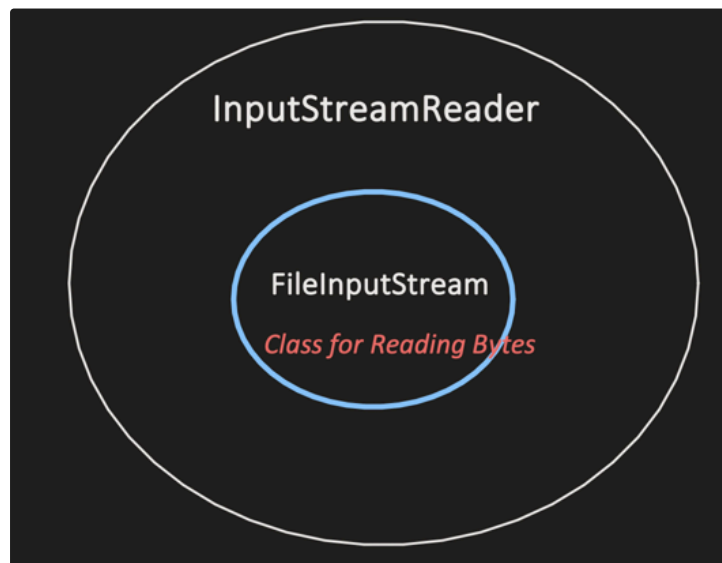
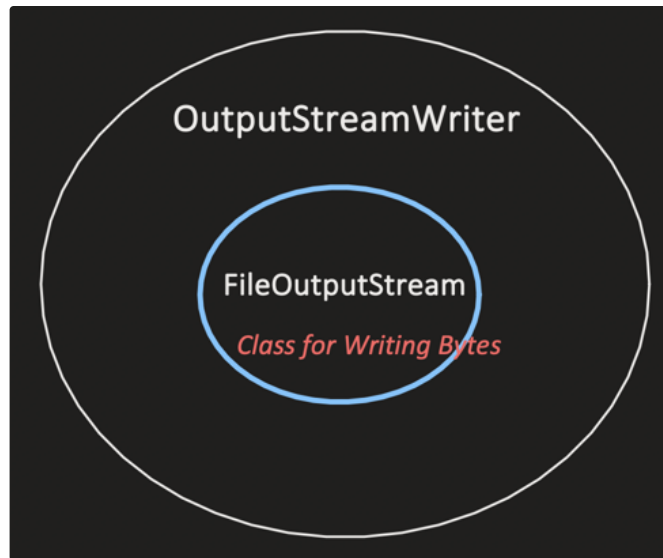
So, if we try to read UTF-8 file using `FileInputStream`, it will read only 1 byte and try to convert it to char,

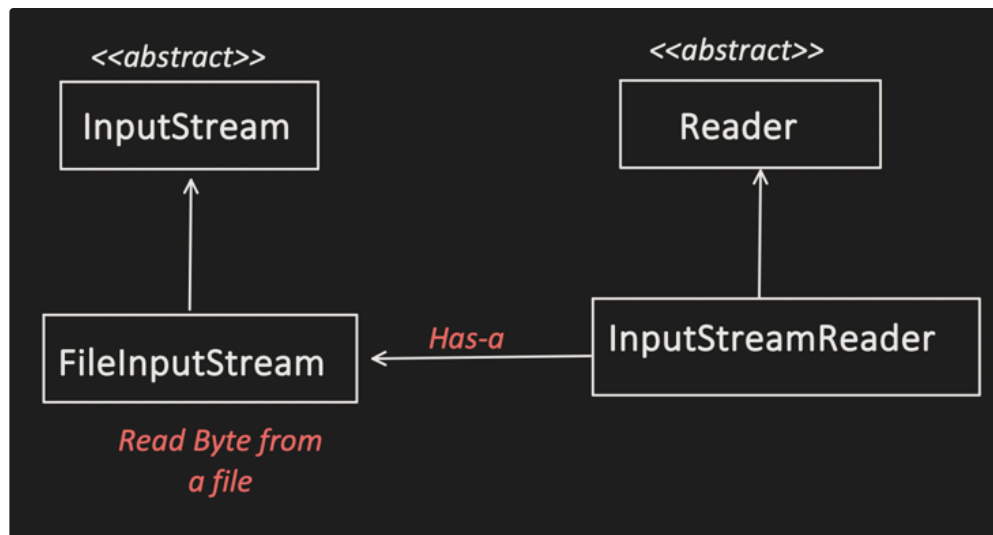
We will get garbage for any non-ASCII character.

Solution: `OutputStreamWriter` / `InputStreamReader`

`OutputStreamWriter`: converts chars → encoding specific format → bytes

`InputStreamReader` : bytes → decoding specific format → chars





```

1 public class CharacterStreams {
2     public static void main(String[] args) throws IOException {
3
4         // Writing text with OutputStreamWriter
5         // We explicitly specify UTF-8 so the file is always written
6         // correctly
7         OutputStreamWriter writer = new OutputStreamWriter(
8             new
9             FileOutputStream("text.txt"),
10            StandardCharsets.UTF_8 // always specify this
11        );
12        writer.write("$"); // multi-byte in UTF-8
13        writer.write("नमस्ते"); // 3 bytes per char in UTF-8
14        writer.close();
15
16        // Reading text with InputStreamReader
17        InputStreamReader reader = new InputStreamReader(
18            new
19            FileInputStream("text.txt"),
20            StandardCharsets.UTF_8
21        );
22        // read() returns ONE character as int, or -1 at end.
23        int ch;
24        /*
25         * - Reader return 1 decoded character.
26         * This decoded character is made up from multiple bytes.
27         */
28        while ((ch = reader.read()) != -1) {
29            System.out.print((char) ch);
30        }
31        reader.close();
32    }
33 }
34
35 }
36

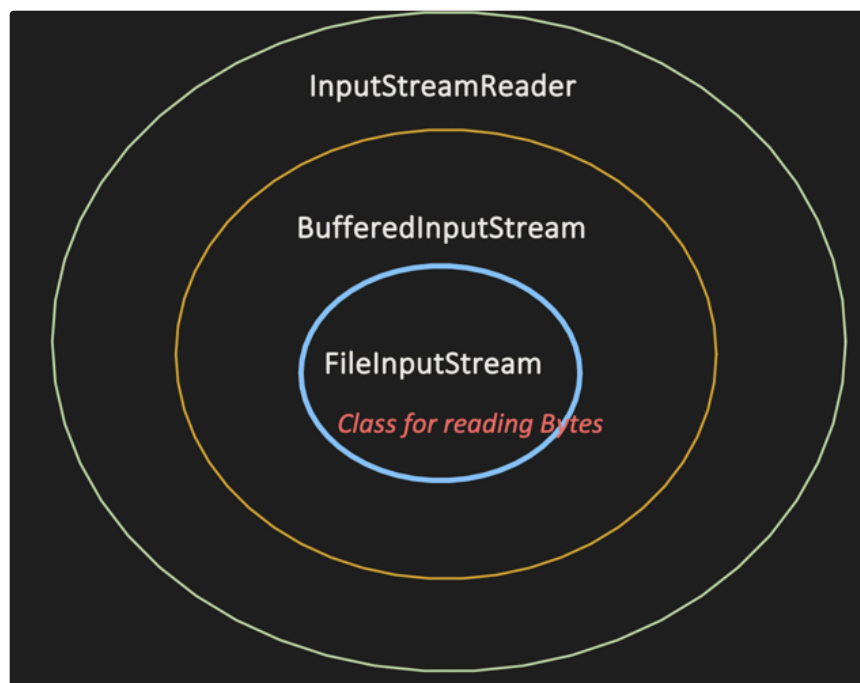
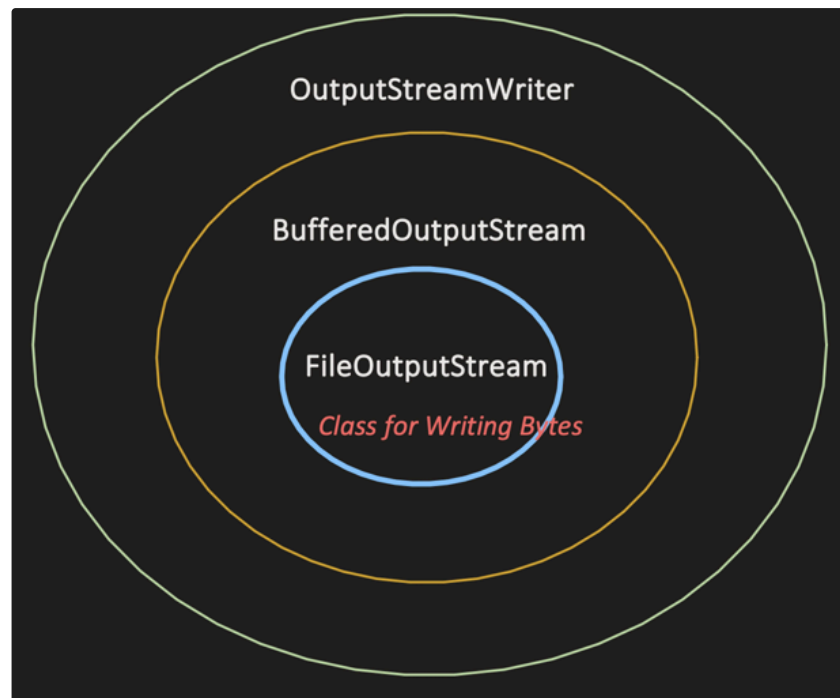
```

Very similar to:

BufferedOutputStream and BufferedInputStream

We have: ***BufferedReader and BufferedWriter***

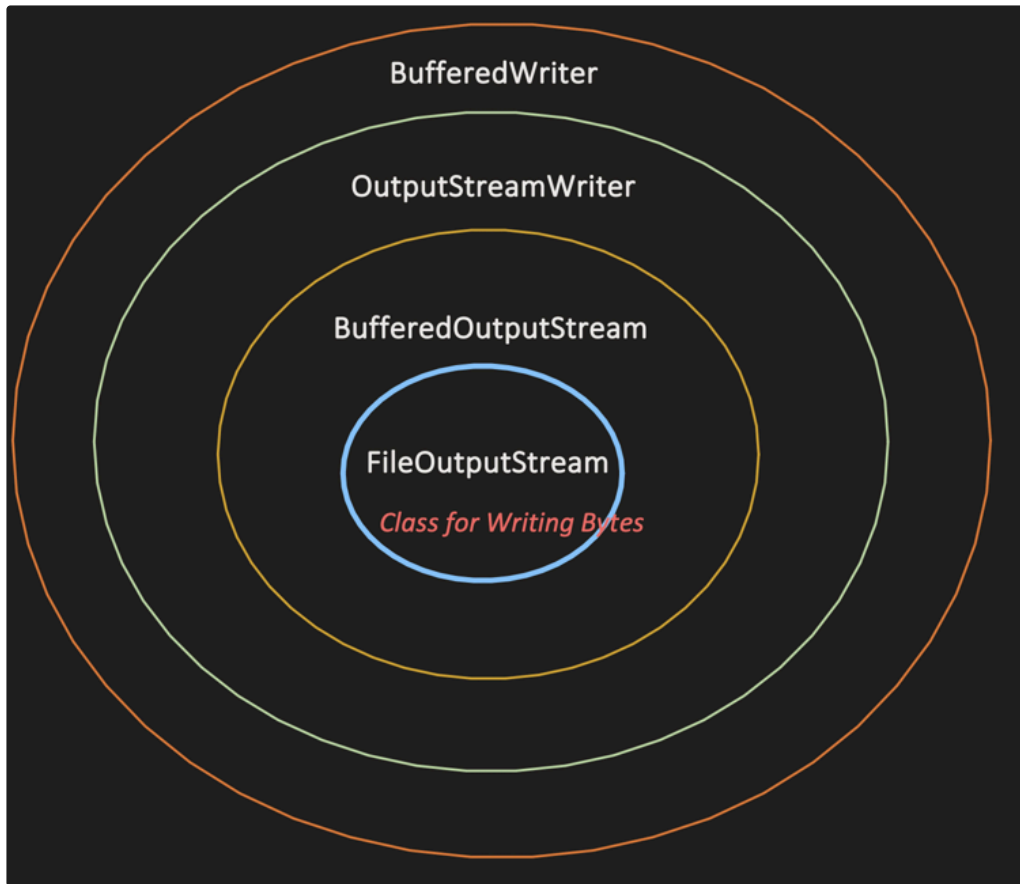
I can have:

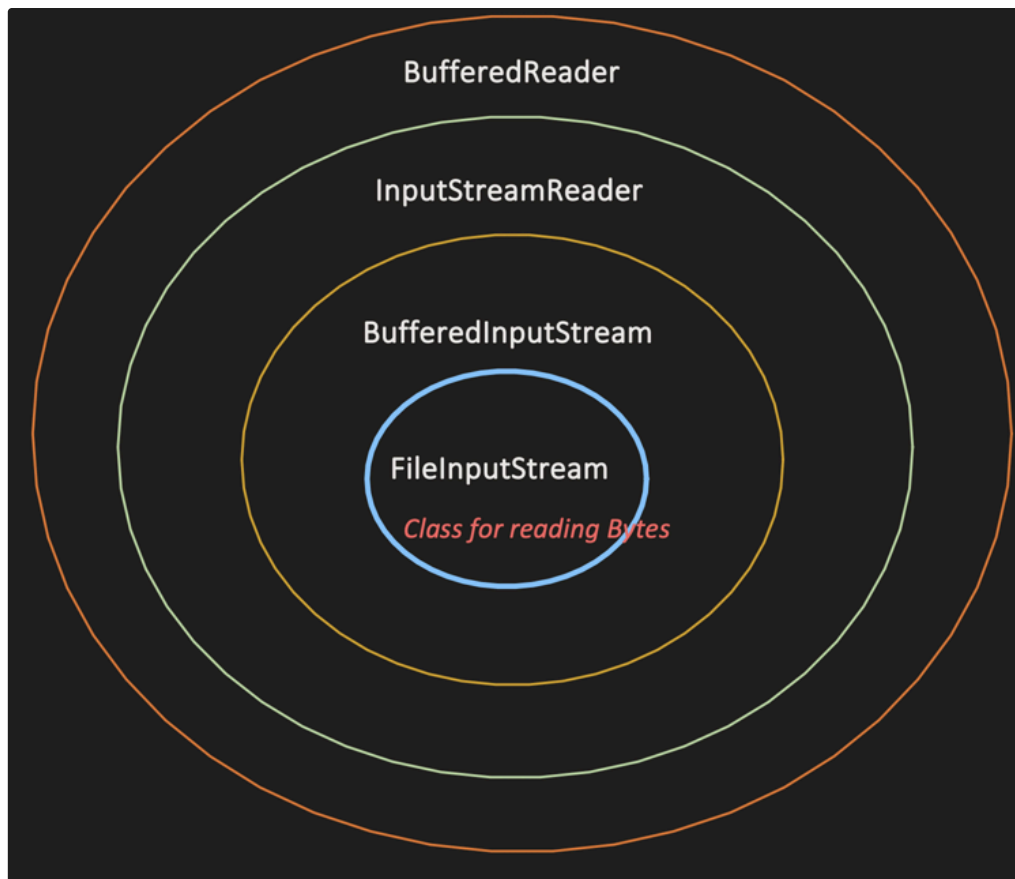


Here lets say, because of **BufferedInputStream**:

- 8192 bytes in 1 syscall will be fetched and buffered.
- Now, `InputStreamReader read()` call, fetches multiple byte do the decoding and return the character.

But this can be made more efficient using ***BufferedReader and BufferedWriter***





Here lets say, because of **BufferedInputStream**:

- 8192 bytes will be fetched and buffered.

But because of **BufferedReader**:

- Stores many decoded chars (which is done by `InputStreamReader`) and stored in buffer in `char[]`.
- And also provide **`.readLine()`** method.

```
1 public class BufferedReadWrite {
2
3     public static void main(String[] args) throws IOException {
4         BufferedWriter bw = new BufferedWriter(new
5         OutputStreamWriter(new BufferedOutputStream(
6         new FileOutputStream("lines.txt"))
7         ,StandardCharsets.UTF_8));
8         bw.write("hello everyone");
9         bw.write("this is Shrayansh");
10        bw.close();
11
12        BufferedReader br = new BufferedReader(new
13        InputStreamReader(new BufferedInputStream(
```

```
13 new FileInputStream("lines.txt"))
14 ,StandardCharsets.UTF_8));
15
16     String line;
17     while ((line = br.readLine()) != null) {
18         // Returns null (NOT -1) when end of stream is reached.
19         System.out.println(line);
20     }
21     br.close();
22 }
23 }
24
```